

Run the Loop: Lab Tracker

Name _____

Every lasting project runs the same loop for every change: **say what you'll change, make it on a branch, explain it, get a second reader, answer them, merge under the project's rules.** Today you run it once, for real, on your own repository. Check off each step and write down the numbers as you go; they're what you'll hand in. **The full steps and every command are on the Canvas page *Git Workflow Lab Guide***; this sheet keeps your place.

Before you start · Can you push?

5 min

- ☐ **Identity set:** `user.name` and `user.email` are right
- ☐ **Fresh copy:** on `main`, pulled
- ☐ **Push works:** `git push --dry-run` says up-to-date
- ☐ **Reviewer has access** (solo: partner added, role Write)
- ☐ My reviewer: _____
- ☐ I review: _____

```
# check, then fix if wrong
git config --global user.name
git config --global user.email
git switch main && git pull
git push --dry-run
```

Part 1 · Your change: issue → branch → commit → pull request

25 min

- ☐ **Task picked** (circle): A design link · B license · C .gitignore · D README fix · E docs index
- ☐ **Issue filed**, assigned to me. Issue # _____
- ☐ **Branched** from fresh `main`: _____
- ☐ **Committed** with a subject *and* a body. Subject passes "If applied, this commit will ____"
- ☐ **Pushed** with `-u`
- ☐ **PR opened**, with `Closes #`. PR # _____
- ☐ **Reviewer requested**

```
git switch -c docs/link-design-doc

# make the change, then look at it
git status
git diff

git add README.md
git commit      # no -m
git log -1
git commit --amend # fix it before pushing

git push -u origin docs/link-design-doc
```

Part 2 · The second reader: review, respond, merge

15 min

- ☐ **Reviewed** PR # _____: two or more comments, one substantive, nits marked `nit`:
- ☐ **Every comment on mine answered**: fixed (new commit, same branch), explained, or moved to a new issue
- ☐ **Approved** by someone who didn't write it
- ☐ **Merged**; squash message edited; branch deleted; **issue closed** itself

```
# after the merge
git switch main
git pull
git log --oneline -5
git branch -D docs/link-design-doc
```

Part 3 · A merge conflict, on purpose

12 min

- ☐ **A and B both branched from the same `main`**, before either merged. I'm: A · B · both (solo)
- ☐ **A changed the README status line** to "Design complete..."; **B added** an issue link to the same line
- ☐ **A's PR merged**; B's now shows a conflict
- ☐ **B resolved it locally by merge**, keeping both changes; pushed
- ☐ **B's PR reviewed and merged**. Conflict PR # _____

```
git switch docs/status-issue-link
git fetch origin
git merge origin/main
# edit README: read both sides,
# keep both, delete <<<<<< ===== >>>>>>
git add README.md
git commit && git push
# lost? start over:
git merge --abort
```

Part 4 · Your project's git standards

12 min · page 2

Decide the six on the back, a reason for each. Then write them into `docs/git-standards.md` and **ship it through the loop**: branch, PR, review, merge, the way it says to.

Hand in (the repository is the submission): your issue, your PR, the review you wrote, the conflict PR, and `docs/git-standards.md` on `main`.

Our Git Standards

Project _____

Every project has rules for how a change gets in. On most, they're never written down, so every newcomer guesses and every maintainer applies them a little differently. **Writing them down is the difference between a norm and a hope.** Each decision below has a suggested default and some reasonable alternatives. **Every one is your project's call.** Choosing differently is fine; what isn't fine is a choice without a reason. Fill in your choice and one line of why, then copy it into `docs/git-standards.md`. That file becomes the heart of your `CONTRIBUTING.md`.

DECISION	SUGGESTED DEFAULT · ALTERNATIVES	OUR CHOICE, AND WHY
Branch names <i>Can a stranger tell what a branch is from its name?</i>	type/short-description (feat/, fix/, docs/, chore/) Or: add the issue number (fix/42-empty-search); initials as a prefix	CHOICE _____ WHY _____ _____
Commit messages <i>Will the log read like a history?</i>	The seven rules Or: the seven rules plus a type prefix, as in Conventional Commits (docs: link the design doc)	CHOICE _____ WHY _____ _____
Pull request size <i>Can it be reviewed in one sitting?</i>	One logical change , small enough to review in one sitting Or: a line-count guideline (split above ~400 changed lines)	CHOICE _____ WHY _____ _____
Review <i>Who must read it before it merges?</i>	One approval from someone who didn't write it Or: two approvals; an owner for certain files. Solo: your review partner, or a self-review checklist when no one's available, said honestly	CHOICE _____ WHY _____ _____
Merge strategy <i>What should main's history look like?</i>	Squash and merge : one commit per change; the steps stay in the PR Or: merge commits (every branch commit kept); rebase and merge (every commit kept, and every one must be clean)	CHOICE _____ WHY _____ _____
"Done" <i>When is a change finished?</i>	Reviewed, merged, runs on main, docs updated if behavior changed Or: add "tests pass" once there are tests; "issue closed with a note"	CHOICE _____ WHY _____ _____

Optional: make GitHub enforce it. Settings → General → Pull Requests: allow only your merge method; if you squash, default the message to *Pull request title and description*; auto-delete head branches. A required approval is a branch ruleset (Settings → Rules). **Solo:** GitHub won't let you approve your own PR, so a required approval means every merge waits on your partner.

The test before you merge it: does the pull request that adds these standards follow them? Branch name, commit message, description, reviewer, merge method. A standard its own PR breaks isn't a standard.

Instructor sign-off before you leave: _____ Anything flagged: _____

CS 301R · Studio I